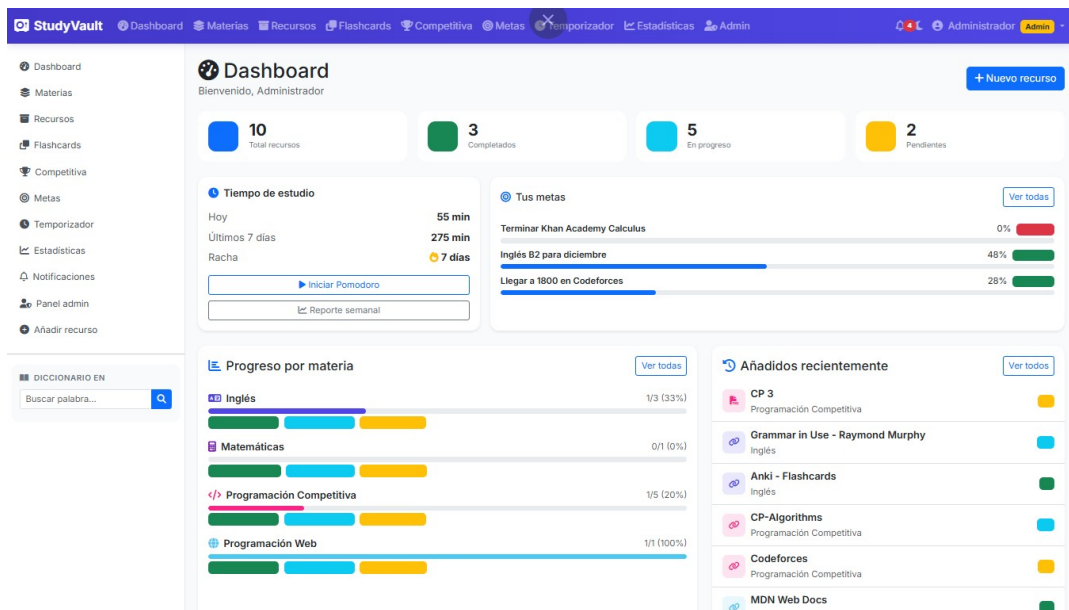


StudyVault

Gestor de material de estudio con repetición espaciada

Cumplimiento de la Rúbrica

Análisis punto por punto frente a `descripcion.md`



Autor: José Roberto García Correa
Materia: Programación Web
Semestre: Sexto
Fecha: Mayo 2026
Repositorio: StudyVault (rama develop)

1. Tabla resumen

Leyenda: ✓ **Cumple** • **Parcial** × **No cumple**

#	Requisito	Estado
Gen	Especificaciones generales (8 puntos)	✓ Cumple
1	Interfaz Web (HTML + CSS)	✓ Cumple
2	Cliente (JavaScript)	✓ Cumple
3	Servidor (PHP OOP)	✓ Cumple
4	Base de datos MySQL	✓ Cumple
5	Autenticación, sesiones y roles	✓ Cumple (Invitado + Admin)
6	Funcionalidades dinámicas obligatorias	✓ Cumple (DataTables + Admin)
7	AJAX / fetch	✓ Cumple
8	Consumo de servicios web (4 APIs)	✓ Cumple
9	Seguridad mínima	✓ Cumple (supera lo pedido)
10	Arquitectura MVC	✓ Cumple
Opc	Tema oscuro	✓ Cumple
Opc	Bitácora de acciones	✓ Cumple
Opc	Recuperación de contraseña	✓ Cumple
Opc	Exportar PDF o Excel	✓ Cumple (CSV + PDF)
Opc	Gráficas estadísticas	✓ Cumple (6 Chart.js)
Opc	Notificaciones	✓ Cumple
Opc	Panel administrativo	• Parcial (mini-panel)

Conclusión: 10 de 10 requisitos obligatorios cumplidos, 6 de 7 mejoras opcionales implementadas, 1 parcial. Sin requisitos no cumplidos.

2. Especificaciones generales

Las 8 especificaciones generales de `descripcion.md` están cubiertas:

- **HTML5 / CSS3** – Bootstrap 5 + `assets/css/app.css` (custom).
- **JavaScript y AJAX/fetch** – `assets/js/app.js` + scripts por vista.
- **PHP orientado a objetos** – `models/`, `controllers/`, autoload con `spl_autoload_register`.
- **PDO con consultas preparadas** – `config/database.php` + todos los modelos.
- **MySQL** – 16 tablas relacionales.
- **Sesiones y seguridad** – sesión segura, CSRF, hashing bcrypt, throttling, cabeceras (CSP).
- **Consumo de servicios web** – 4 APIs públicas integradas vía proxy PHP.
- **Diseño responsivo** – Bootstrap responsive + modo oscuro.

3. Requisitos mínimos (10 puntos)

3.1 Interfaz Web – ✓ Cumple

Responsive, navbar + sidebar, formularios estructurados, CSS externo (`app.css`), tablas estilizadas, componentes modernos. Bootstrap y Font Awesome incluidos. DataTables aprovechado para ordenamiento por columna.

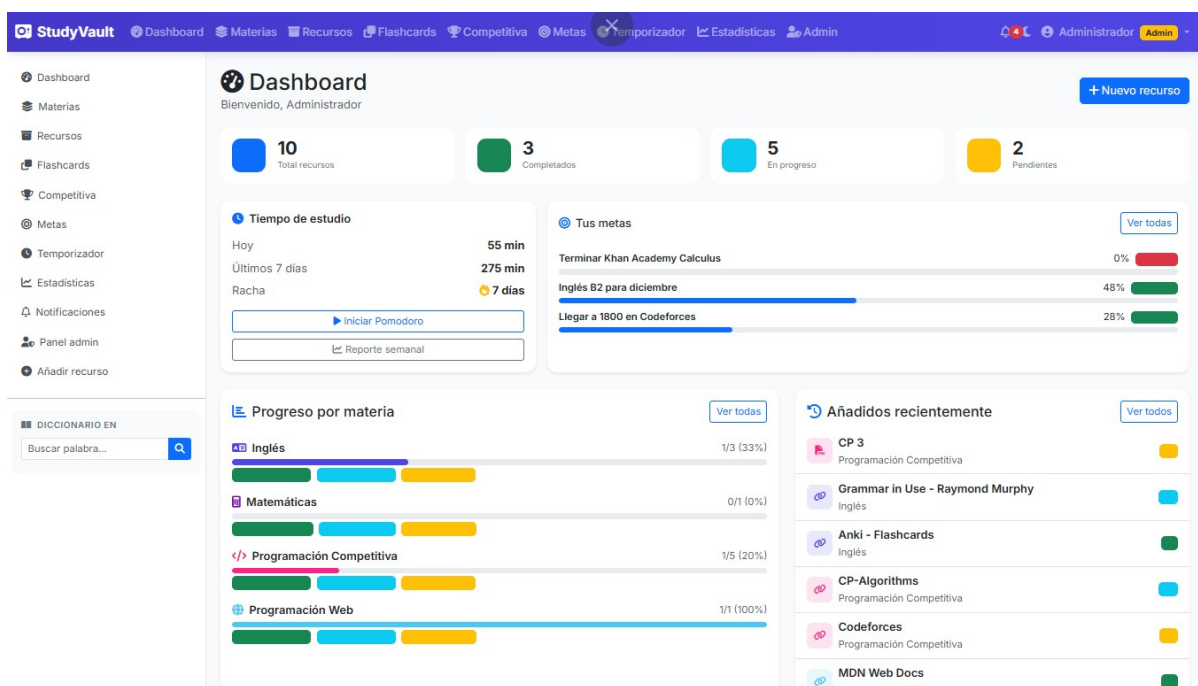


Figura 1: Dashboard administrativo con tarjetas de stats, tiempo de estudio, racha y metas.

3.2 Cliente (JavaScript) – ✓ Cumple

Validaciones dinámicas (formularios, coincidencia de contraseñas), eventos, manipulación del DOM, fetch, actualización parcial sin recargar, búsquedas y filtros instantáneos (Recursos), alertas dinámicas, modo oscuro persistente.

3.3 Servidor (PHP OOP) – ✓ Cumple

MVC con clases y métodos, **inclusión modular** (`config/init.php` + autoloader), **reutilización** (motor SM-2 compartido entre flashcards y CP). CRUDs completos (materias, recursos, unidades, metas, flashcards, problemas CP, plantillas). Validación en servidor, manejo de errores centralizado (`set_exception_handler` + log), sanitización (`htmlspecialchars` + preparadas).

3.4 Base de datos MySQL – ✓ Cumple

Modelo relacional (16 tablas), PK/FK, relaciones 1:N y N:M (`goal_resources`), integridad referencial (ON DELETE CASCADE), índices. PDO + consultas preparadas obligatorias.

3.5 Autenticación, sesiones y roles – ✓ Cumple

Login, logout, sesiones, roles `admin/student`, `password_hash/password_verify`, protección de rutas (`requireLogin/requireAdmin`). Supera el mínimo con CSRF, throttling y regeneración de sesión.

Roles diferenciados:

- **Admin:** accede al mini-panel administrativo (gestión de usuarios + uso global).
- **Student:** usuario regular.
- **Invitado:** sin sesión, puede explorar plantillas públicas en modo solo lectura.

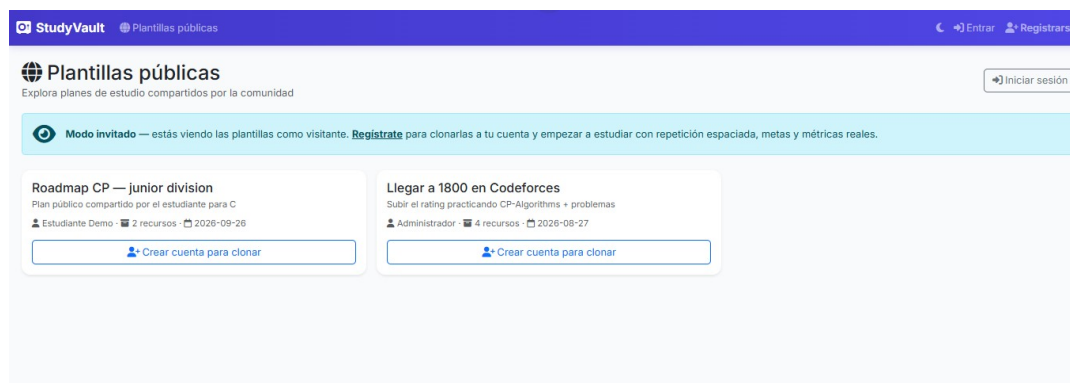


Figura 2: Rol **Invitado**: el navbar se simplifica, sin acceso a las funciones privadas. Sólo puede explorar plantillas públicas y registrarse para clonarlas.

3.6 Funcionalidades dinámicas obligatorias – ✓ Cumple

Paginación, búsquedas, filtros, CRUD, subida de archivos (PDF/imágenes, máx. 50 MB), dashboard, **ordenamiento por columna** con DataTables en Recursos / Flashcards / Competitiva / Admin, y panel administrativo con activar/desactivar usuarios.

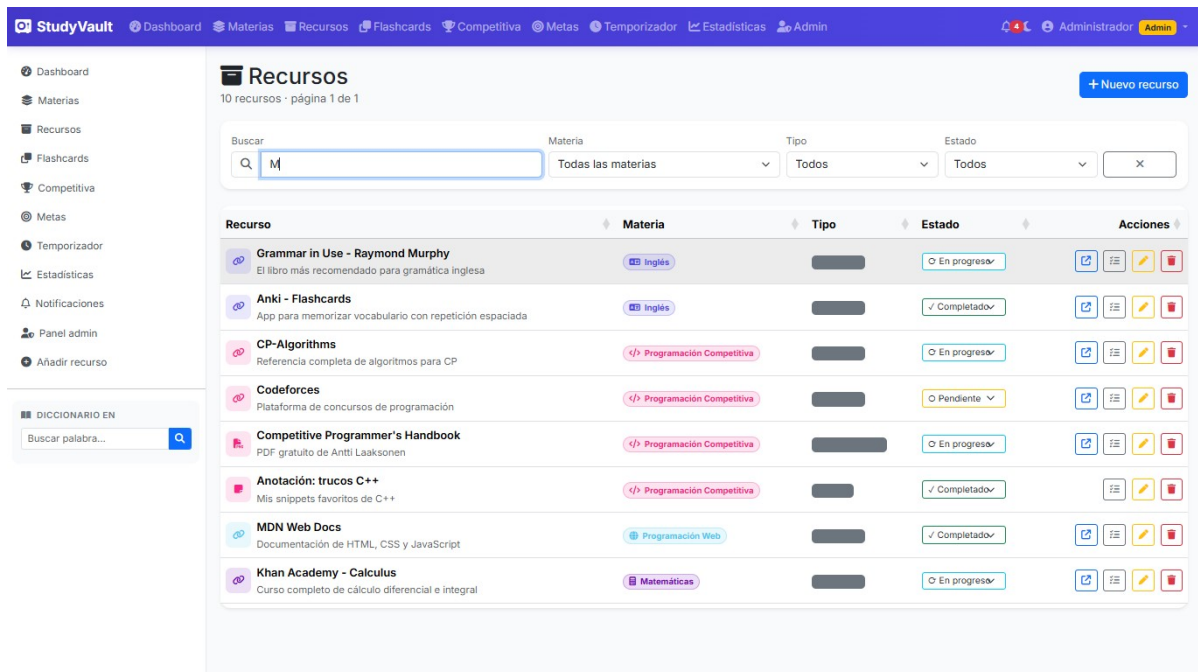


Figura 3: Listado de Recursos con búsqueda en vivo, filtros por materia/tipo/estado, paginación server-side y ordenamiento por columna.

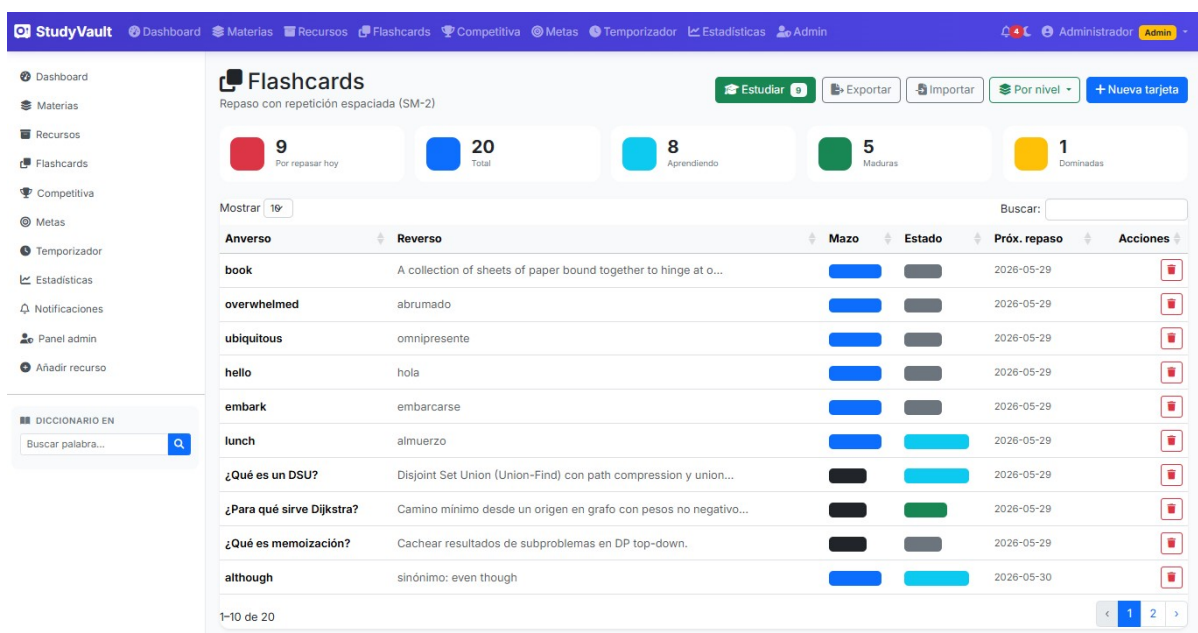


Figura 4: Tabla de Flashcards con DataTables activo (encabezados ordenables) y tarjetas de estadísticas por estado SM-2.

3.7 AJAX / fetch – ✓ Cumple

Consultas automáticas, actualización de tablas (búsqueda en vivo de Recursos), eliminación sin recargar, formularios dinámicos (modales), búsqueda en tiempo real, diccionario, repaso de flashcards / CP, centro de notificaciones, panel admin. Uso extenso.

3.8 Consumo de servicios web – ✓ Cumple (supera el mínimo)

4 APIs públicas gratuitas, sin API key:

API	Uso	Endpoint
Free Dictionary	Definición, fonética, audio	dictionaryapi.dev
Datamuse	Colocaciones (uso real)	api.datamuse.com
Tatoeba	Frases de ejemplo reales	tatoeba.org
Codeforces	Rating, problemas, concursos	codeforces.com/api

Todas se consumen vía proxy PHP (evita CORS + caché + rate limiting).

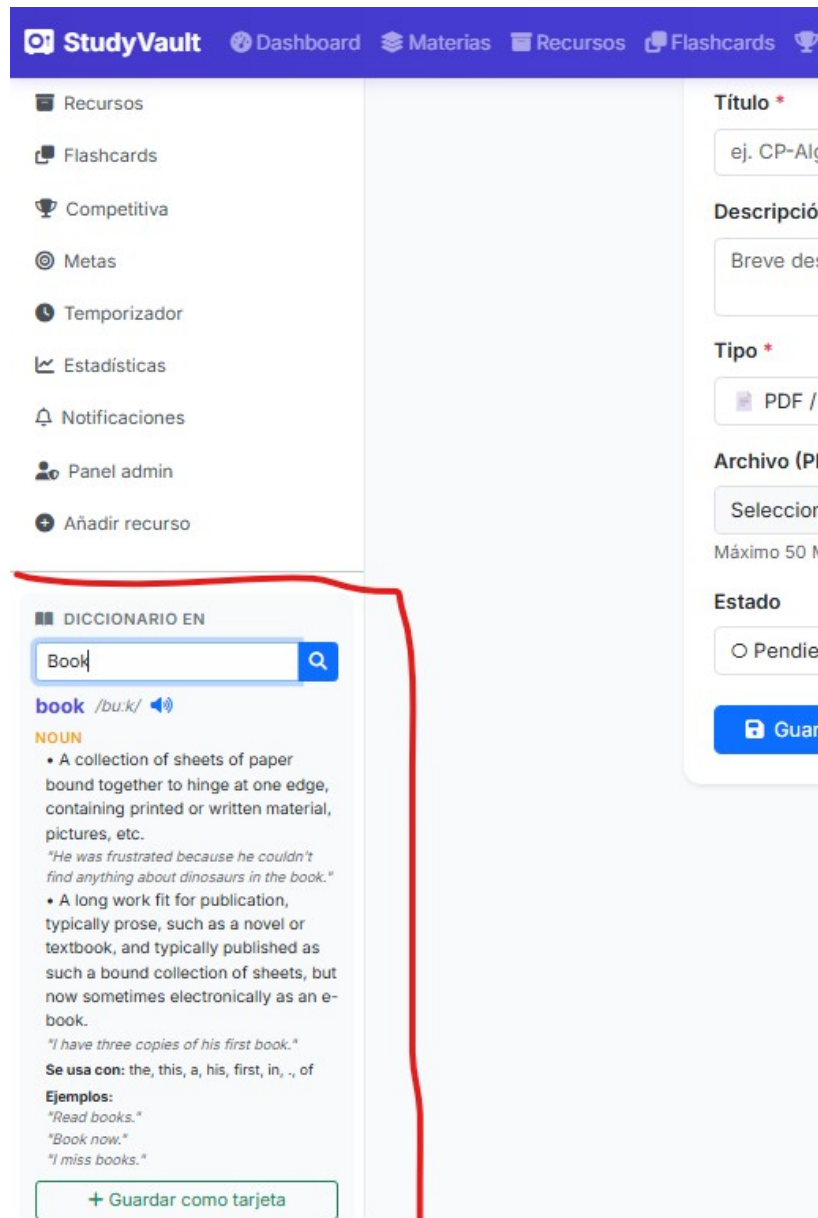


Figura 5: Diccionario integrado en la barra lateral: definición (Free Dictionary) + “se usa con” (Datamuse) + frases de ejemplo (Tatoeba), con botón “Guardar como tarjeta”.

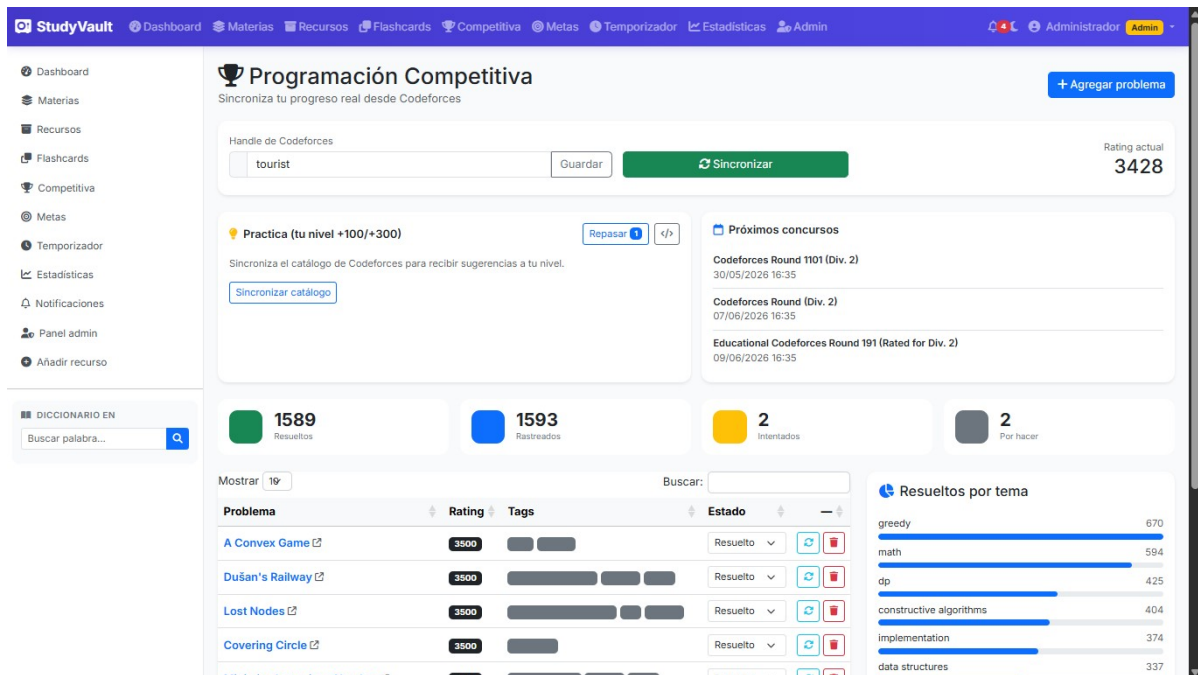


Figura 6: Panel de Programación Competitiva: handle de Codeforces sincronizado, rating, problemas importados, análisis por tag y sugerencias de práctica.

3.9 Seguridad mínima – ✓ Cumple (supera lo pedido)

- **Validaciones cliente/servidor:** en cada formulario.
- **Anti SQL Injection:** PDO preparado en TODAS las queries.
- **Anti IDOR:** cada query filtra por `user_id` – tests automatizados en `tests/security_test.php`.
- **Sanitización:** `htmlspecialchars()` en salidas.
- **Extras:** CSRF en todo POST, `password_hash` (bcrypt), `session_regenerate_id`, cabeceras (X-Frame-Options, nosniff, Referrer-Policy, CSP), rate limiting, throttling de login, `.htaccess` que bloquea ejecución en uploads/.

3.10 Arquitectura MVC – ✓ Cumple

`config/`, `models/`, `controllers/`, `views/`, `assets/{css,js,img}` + `api/` para los endpoints AJAX. Coincide con la estructura recomendada.

Listing 1: Estructura de carpetas

```
studyvault/
  config/      init.php  database.php  config.php
  models/      User, Resource, Subject, Flashcard, CpProblem, Goal, ...
  controllers/ AuthController, ResourceController, StatsController, ...
  views/       auth/ dashboard/ flashcards/ cp/ goals/ admin/ ...
  api/         dictionary.php datamuse.php tatoeba.php search.php ...
  assets/      css/ js/ vendor/ uploads/
  tests/       sm2_test.php goal_test.php security_test.php ...
```


4. Funcionalidades opcionales (mayor calificación)

4.1 Nivel intermedio

4.1.1. Recuperación de contraseña – ✓ Cumple

Flujo con token: tabla `password_resets` (token de 32 bytes hex, expiración 1h, marca `used` al canjearse). Como XAMPP no tiene SMTP, el enlace se muestra en pantalla en modo demo.

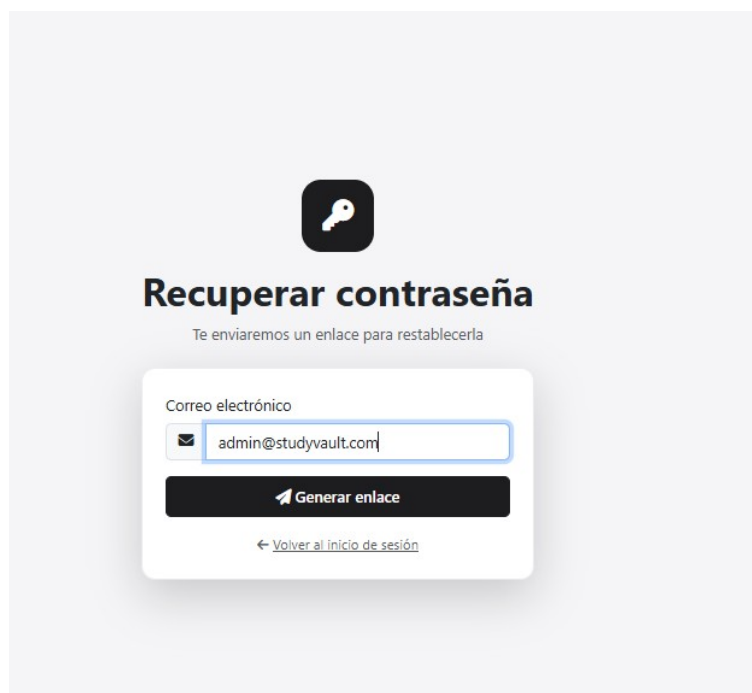


Figura 7: Formulario “Recuperar contraseña” al que se accede desde el login.

4.1.2. Exportar PDF o Excel – ✓ Cumple

CSV de flashcards compatible con Anki, PDF del reporte semanal y de las metas vía `window.print()` + CSS `@media print` (sin dependencias).

4.1.3. Gráficas estadísticas – ✓ Cumple

Chart.js (servido localmente desde `assets/vendor/`), 6 gráficas en `?page=stats`:

1. Minutos de estudio por día (barras, 30 días).
2. Tarjetas por estado SM-2 (doughnut).
3. Vocabulario por nivel CEFR (barras horizontales).
4. Problemas CP por tag (radar – mapa de debilidades/fortalezas).
5. Curva de rating de Codeforces (línea).
6. Histograma de problemas resueltos por dificultad.

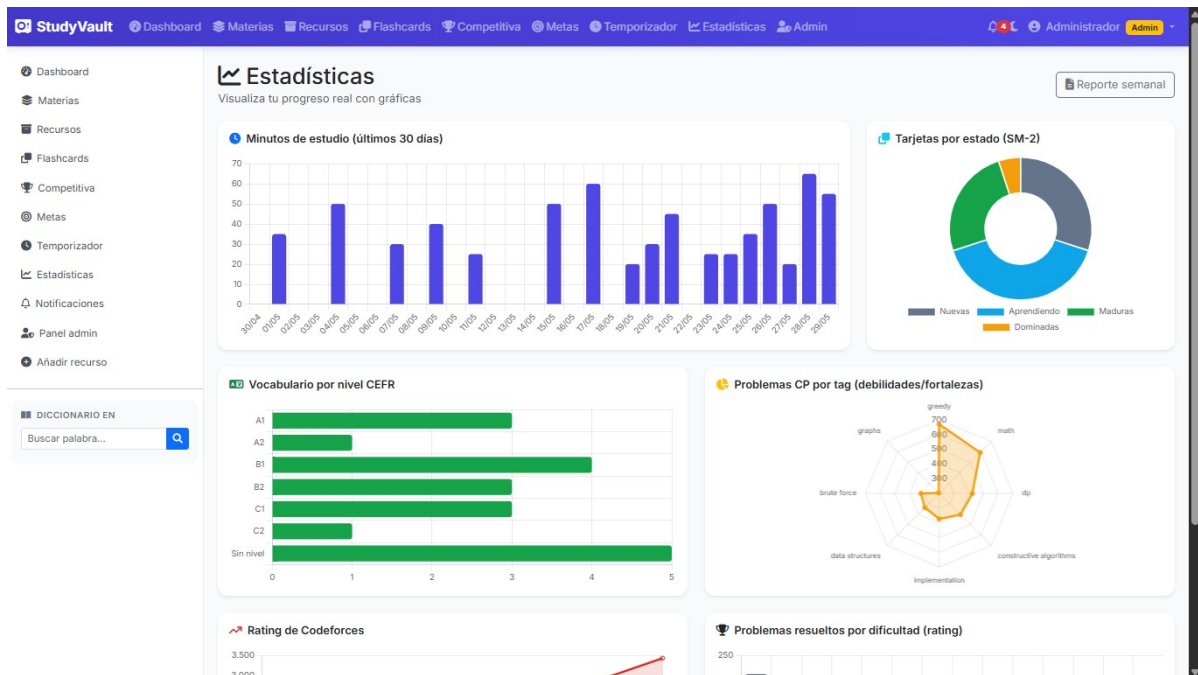


Figura 8: Panel de Estadísticas con las 6 gráficas Chart.js renderizando datos reales.

4.1.4. Notificaciones – ✓ Cumple

Centro in-app con badge en navbar, dropdown auto-actualizable y página completa `?page=notifications`. Generación idempotente por día con 5 reglas: repasos de flashcards, repasos de CP, metas atrasadas, meta diaria sin cubrir, racha en riesgo. Opt-in para Notification API del navegador.

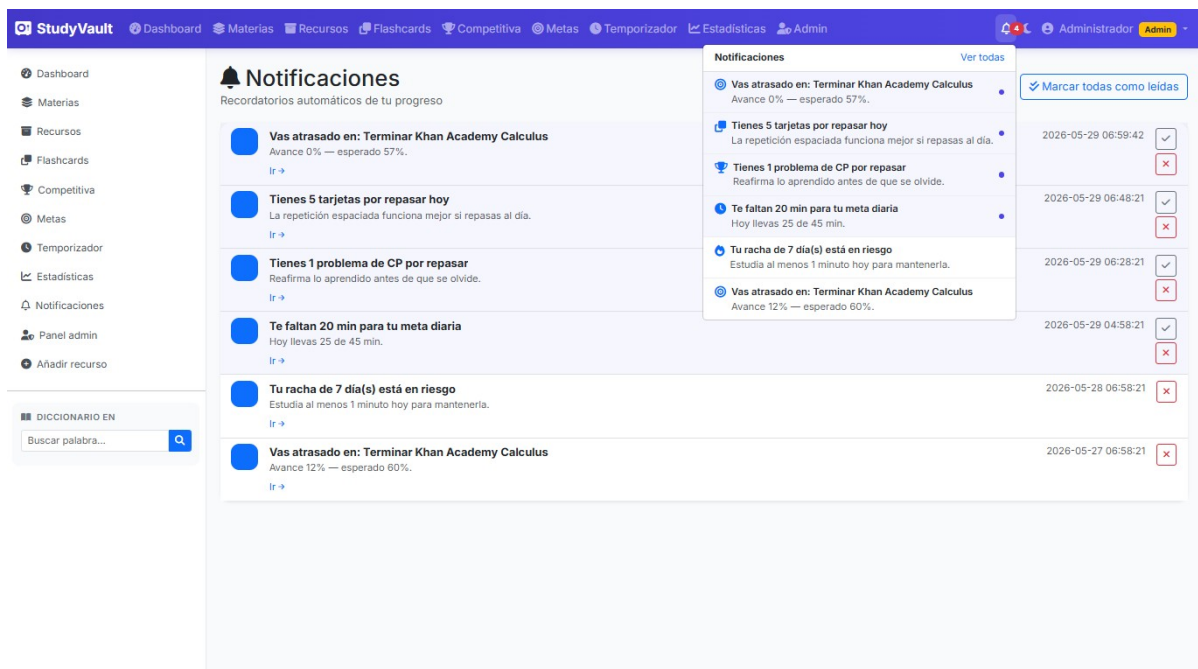


Figura 9: Centro de notificaciones: campana del navbar con dropdown desplegado mostrando los avisos generados automáticamente.

4.1.5. Tema oscuro – ✓ Cumple

Toggle persistente en `localStorage` usando variables CSS bajo `[data-theme="dark"]`.

4.1.6. Bitácora de acciones – ✓ Cumple

Tabla `activity_log` + helper `log_activity()`.

4.2 Nivel avanzado (opcionales no requeridos)

4.2.1. Panel administrativo – • Parcial

Implementado un mini-panel con: listado paginado de usuarios con búsqueda, activar / desactivar, 8 tarjetas de totales globales (usuarios, materias, recursos, flashcards, problemas CP, metas, sesiones, minutos). Falta auditoría granular por usuario.

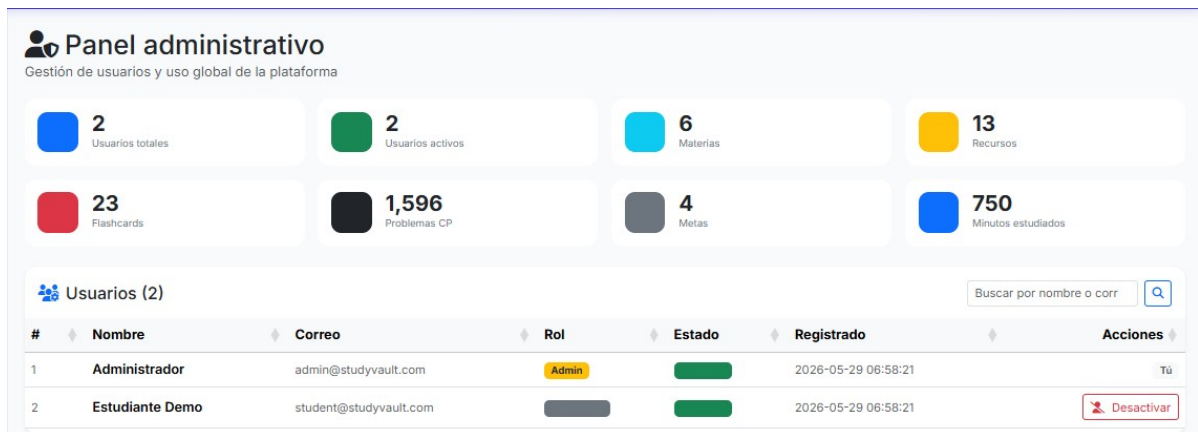


Figura 10: Panel administrativo: gestión de usuarios con búsqueda + estado activo/inactivo + métricas globales de la plataforma.

4.2.2. API REST propia – ✗ No cumple

Existen endpoints JSON internos (`api/*.php`) pero no una API REST documentada con versiones.

4.2.3. WebSockets / Chat – ✗ No cumple

No aplica al propósito (estudio individual).

4.2.4. Docker / Framework PHP – ✗ No cumple

Mejoras de despliegue/escala, opcionales.

5. Entregables

- **Documentación:** `README.md`, `MANUAL_USUARIO.md`, `DESCRIPCION_PROYECTO.md`, `PLAN_IMPLEMENTACION.md`, `GUION_EXPOSICION.md`.
- **Código fuente:** organizado en MVC, comentado.
- **Base de datos:** `ENTREGA/studyvault_completo.sql` – esquema canónico + datos demo en un solo archivo.
- **Manual de usuario:** cubre todas las funciones.

6. Métricas finales del proyecto

Métrica	Valor
Tablas en la BD	16
Modelos PHP (OOP)	12
Controladores	11
Vistas	26
Proxies de API externa	5
APIs externas integradas	4
Pruebas automatizadas (assert)	53
Gráficas Chart.js	6
Migraciones documentadas (v2–v8)	7
Líneas SQL de esquema canónico	~460

Documento generado para la entrega final del proyecto de Programación Web.